TECHNICAL SUMMARY SHEET

A C-Program API that is easy to use and fail-safe for applications to generate log messages.

This C-Program API is free to use under the MIT License agreement.

Technical Advantage:

The primary benefit using this fail-safe C-Program API library over other application loggers, is that it does extensive runtime validation to prevent the application from crashing due to common programmers printf() style pitfalls or unpredictable execution segmentation faults. (e.g., string type %s using NULL, 0 or negative value).

During runtime, the API decode, analyze and apply 18 different Bound Parameter verification techniques to ensure log messages are used accurately, fail-safe and reliable to generate.

In addition, if a runtime validation fails the API generates an error code and simply returns, allowing the application to continue normal processing (no need for the application to check/stop processing)..

API Usage:

Provides a simple printf() style function interface commonly used to generate application log messages:

```
WandiSSAL_LoggerAppsMsg( DevProd, LogicalName, SeverityLevel, "Format string specification, string specifier value %s",  ”string value” );
```

1. DevProd controls generating development and/or production log messages.
2. LogicalName indicates file name, function name or common name across multiple files.
3. SeverityLevel indicates different levels of severity importance.
4. The remainder of the arguments are similar to printf() style specification.
5. Uses a runtime Bound Value Validation method to ensure fail-safe application execution:
    a. Use only LogicalName defined for use by the API.
    b. Use only SeverityLevel defined for use by the API.
    c. Decode printf() format string specification for string specifiers and match to corresponding log message arguments.
    d. Validate length of format string specification to prevent use of unbounded strings.
    e. Validate length of string argument to prevent the use of improper NULL terminated strings.
    f. Validate specific ASCII characters to avoid use of non-printable characters in log message.
    g. Validate against an acceptable set of format string specifiers to prevent unsafe and malicious use of attacker string specifiers.
    h. Validate LogicalName and SeverityLevel for supported values.
    i. Prevent use of invalid NULL, 0, negative values for %s format string specifier.
    j. Block the use of untrusted or malicious format string specifiers.
    k. Block the use of ill-formed or non-standard format specifiers.
    l. Application does not need to check API return status and exist when a validation fails.
    m. Most fail-safe occurrence will not impact application execution flow.

    Complete list and explanation on how the Bound Value Validation works is provided in WandiSSAL-Bound-Capabilities.pdf and detailed programming examples are listed the WandiSSAL-examples.c program.
6. Standard log file name includes: [date.timestamp.sequence-number].log
7. Standard output log file content format:

02/09/2026 12:01:55 [3650:sample_program] [DATABASE] [INFO] Message that generates 1 string and 2 integer arguments [String Argument], [98] and [99]

where:

“02/09/2026 12:01:55” date and timestamp when the log message was generated.
“[3650:sample_program]” application process id and name.
“[DATABASE]” logical message group for related application log messages.
“[INFO]” severity for log message.

The remainder of the output is derived from the C-Programming Language printf() format string specification and corresponding arguments.

Contact: bwoodley@istech.com
Documentations and Download: https://github.com/wandi-ssal/fail-safe-application-logger